SAÉ 2.01 : Développement d'une application – Projet Briqu'IUTO – partie 1

Compétence ciblée :

- Développer - c'est-à-dire concevoir, coder, tester et intégrer - une solution informatique pour un client.

Descriptif générique :

En partant d'un besoin exprimé par un client, l'objectif est de formaliser les besoins, proposer une conception, implémenter et tester son développement.

Les livrables attendus :

– Diagrammes d'analyse et de conception (diagramme de classe + diagrammes de séquence les plus pertinents dont ceux ci-dessous :
- rechercherBoitesParTheme(),
- composerBoitePersonnalisee()

– Code de l'application
– Jeu d'essais prouvant le respect des spécifications
– Rapports collectif et individuels (séparés)

Apprentissages critiques :

– AC11.01 | Implémenter des conceptions simples
– AC11.02 | Élaborer des conceptions simples
– AC11.03 | Faire des essais et évaluer leurs résultats en regard des spécifications

Ressources mobilisées et combinées :

– R2.01 | Développement orienté objets
– R2.03 | Qualité de développement
– R2.13 | Communication technique

Sujet : rappel contexte commun SAE

Un collectionneur passionné de LEGO® souhaite développer une application permettant de gérer et explorer une base de données contenant les informations relatives aux boîtes LEGO®, à leur contenu et à leur organisation par thèmes.
Vous disposez déjà, grâce à la SAE 2.04, d'une base de données relationnelle correspondant au MCD issu du site Rebrickable®.

Cette base permet notamment de modéliser :

- Les **boîtes** (numéro, nom, année, thème)
- Les **thèmes** (thèmes principaux et sous-thèmes)
- Les **contenus** des boîtes
- Les **pièces** (avec catégorie et couleur)
- Les **figurines**
- Les relations de composition :

- pièces contenues dans une boîte
- figurines contenues dans une boîte
- boîtes incluses dans d'autres boîtes (packs)
- gestion des pièces supplémentaires

L'objectif est désormais de développer l'application Java **briquesCollectionManager**, permettant d'exploiter cette base de données via une application en ligne de commande.

Toutefois, vous veillerez à ce que l'application fonctionne avec des données stockées dans des SDD en mémoire avant de vous lancer dans les échanges avec la base de données.

L'analyse devra être réalisée en UML et l'application développée en Java en respectant une architecture orientée objet claire (couches métier / accès aux données / interface).

DEMANDES

1. Liste non exhaustive et minimale de fonctionnalités à implémenter

Utilisateur (collectionneur)

- Rechercher une boîte par numéro ou par nom.
- Consulter le détail complet du contenu d'une boîte :
  - pièces (nom, catégorie, couleur, quantité, supplément ou non)
  - figurines
  - boîtes incluses (dans le cas d'un pack).
- Explorer les boîtes par thème et sous-thème.
- Afficher les statistiques d'une boîte (nombre total de pièces, répartition par couleur, etc.).
- Rechercher les boîtes qui contiennent une pièce précise.
- Ajouter une boîte à sa collection personnelle.
- Indiquer qu'une boîte est complète ou incomplète.
- Générer la liste des pièces manquantes d'une boîte incomplète.
- Composer une boîte personnalisée à partir d'une sélection de pièces existantes.

Administrateur (gestion des données)

- Ajouter une nouvelle boîte dans la base.
- Ajouter une nouvelle pièce et l'associer à une catégorie.
- Créer un nouveau thème ou sous-thème.
- Mettre à jour le contenu d'une boîte (ajout de pièce, figurine ou sous-boîte).

2. Contraintes : fonctionnalités supplémentaires

Recherche intelligente par thème

L'application devra permettre d'afficher toutes les boîtes appartenant à un thème donné, en incluant les sous-thèmes éventuels.

La méthode devra avoir le profil :

public List<Boite> rechercherBoitesParTheme(Theme theme)

Elle devra être correctement testée et documentée comme toutes les méthodes de la SAE.

Composition personnalisée

L'utilisateur pourra créer virtuellement une nouvelle boîte personnalisée en sélectionnant un ensemble de pièces existantes dans la base.

La méthode devra avoir le profil :

public Boite composerBoitePersonnalisee(String nom, List<Piece> pieces)

Cette fonctionnalité devra :

- Vérifier l'existence des pièces dans la base
- Vérifier qu'une boîte identique n'a pas déjà été créée et demander confirmation
- Calculer le nombre total de pièces
- Générer un identifiant unique
- Permettre l'enregistrement en base

Elle devra être correctement testée et documentée comme toutes les méthodes de la SAE.

Interface utilisateur

- L'application devra proposer en lignes de commande des menus et sous-menus organisés.
- La gestion des exceptions devra être correctement traitée (erreurs de saisie, identifiants inexistants, incohérences).

3. Contraintes techniques obligatoires

Vous devrez impérativement, lors de cette SAE, mettre en pratique les notions suivantes :

- Connexion à une base de données relationnelle (JDBC).
- Mise en place de tests unitaires (JUnit).
- Utilisation d'un dépôt GIT avec historique propre.
- Respect des bonnes pratiques de programmation :
    - encapsulation
    - gestion des exceptions
    - JavaDoc
    - architecture claire

4. Éléments attendus dans l'analyse UML

Le diagramme de classes devra notamment faire apparaître :

- Boite (Il faudra distinguer les boites composées et les boites personnalisées qui hériteront de Boite)
- Theme
- Contenu
- Piece
- Categorie
- Couleur
- Figurine
- Relations de composition multiples

Rappel : les diagrammes de séquence devront illustrer **au minimum** :

- rechercherBoitesParTheme()
- composerBoitePersonnalisee()

Annexe : MCD

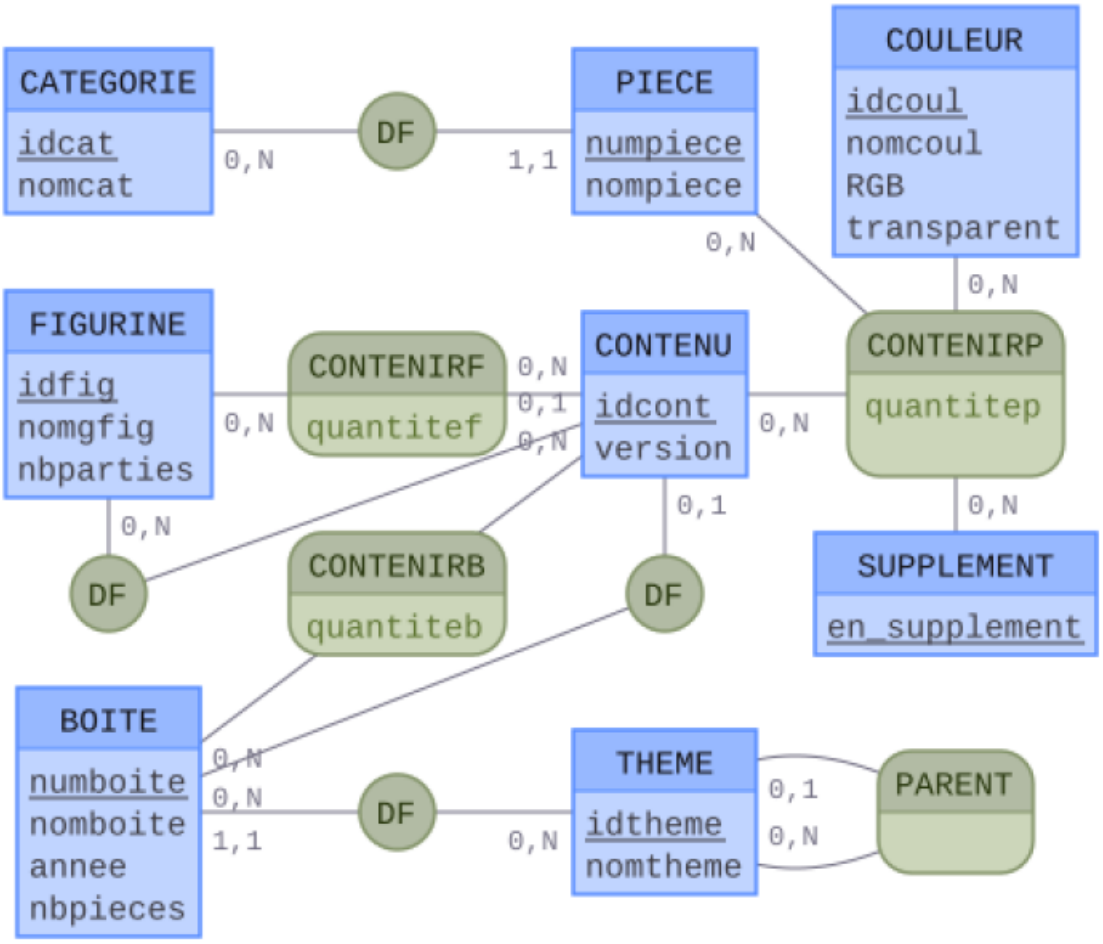

FIGURE 1 – MCD de la base de données des Lego®